Chapter ML:III

III. Linear Models

- Logistic Regression
- Overfitting
- Regularization

©STEIN 2005-2020

Logistic Regression

Binary Classification Problems

Setting:

- The feature space $X \subseteq \mathbf{R}^p$ is an inner product space.
- $C = \{0, 1\}$ is a set of two classes. Similarly: $\{-1, 1\}$, $\{\ominus, \oplus\}$, $\{\text{no}, \text{yes}\}$, etc.
- $c : X \to C$ is the (unknown) ideal classifier for X.
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate c, which is implicitly given via D, with a logistic model function.

©STEIN 2005-2020

Logistic Regression

Binary Classification Problems

Setting:

- The feature space $X \subseteq \mathbf{R}^p$ is an inner product space.
- $C = \{0, 1\}$ is a set of two classes. Similarly: $\{-1, 1\}$, $\{\ominus, \oplus\}$, $\{\text{no}, \text{yes}\}$, etc.
- $c : X \to C$ is the (unknown) ideal classifier for X.
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate c, which is implicitly given via D, with a logistic model function.

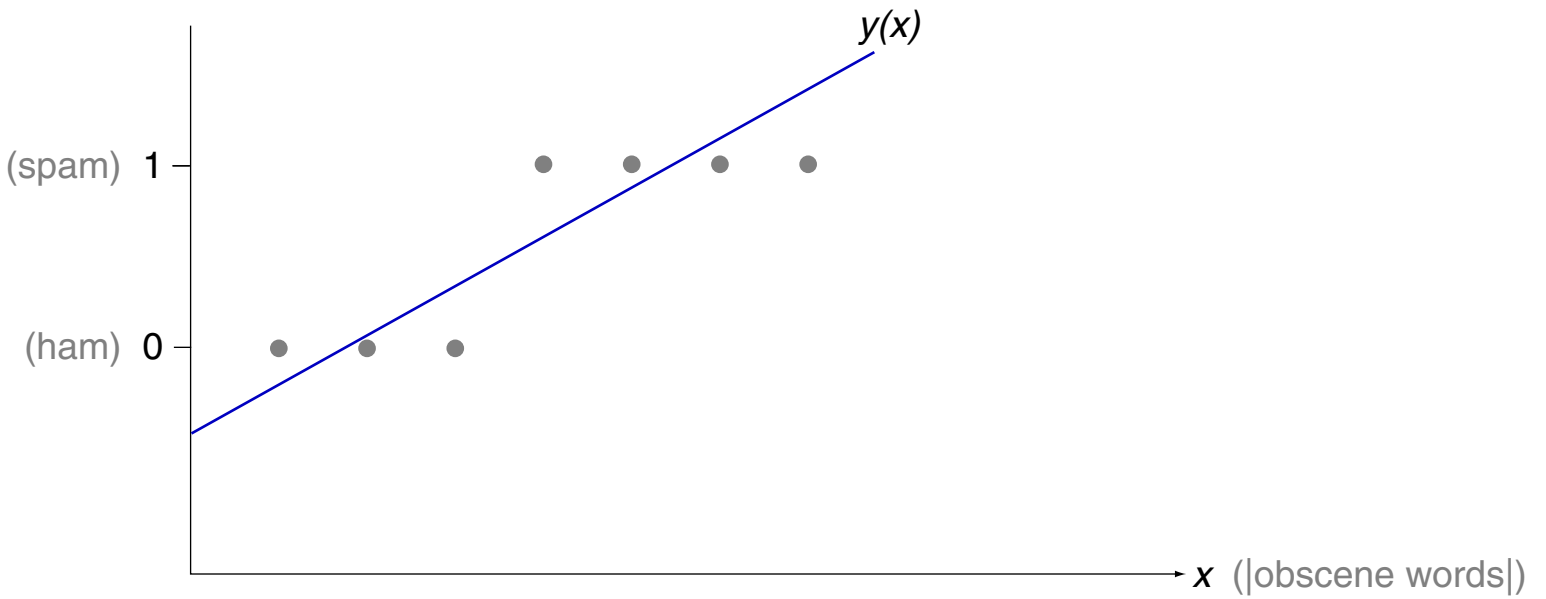
Examples for binary classification problems:

- Is an email spam or ham?
- Is a patient infected or healthy?
- Is a bank customer creditworthy or not?

©STEIN 2005-2020

Logistic Regression

Linear Regression

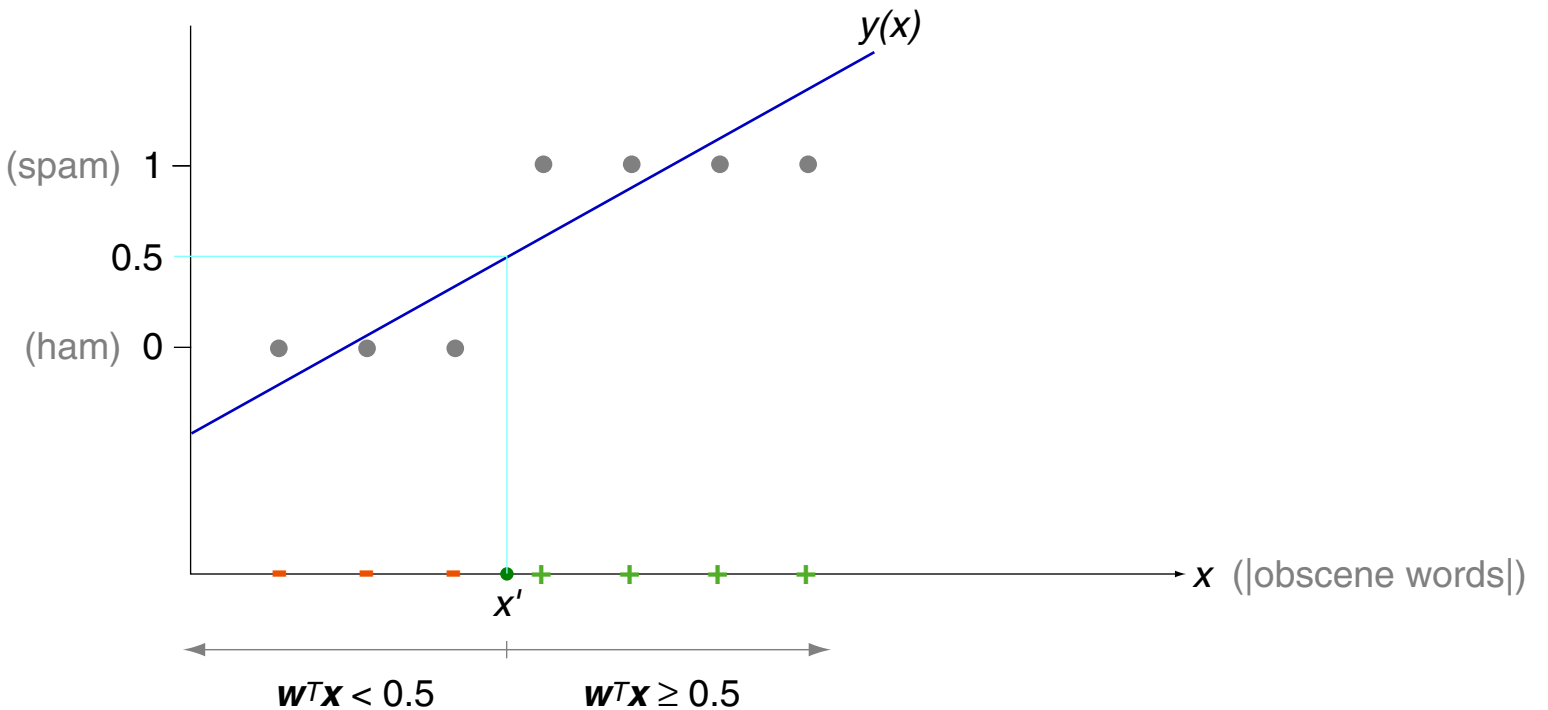


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)

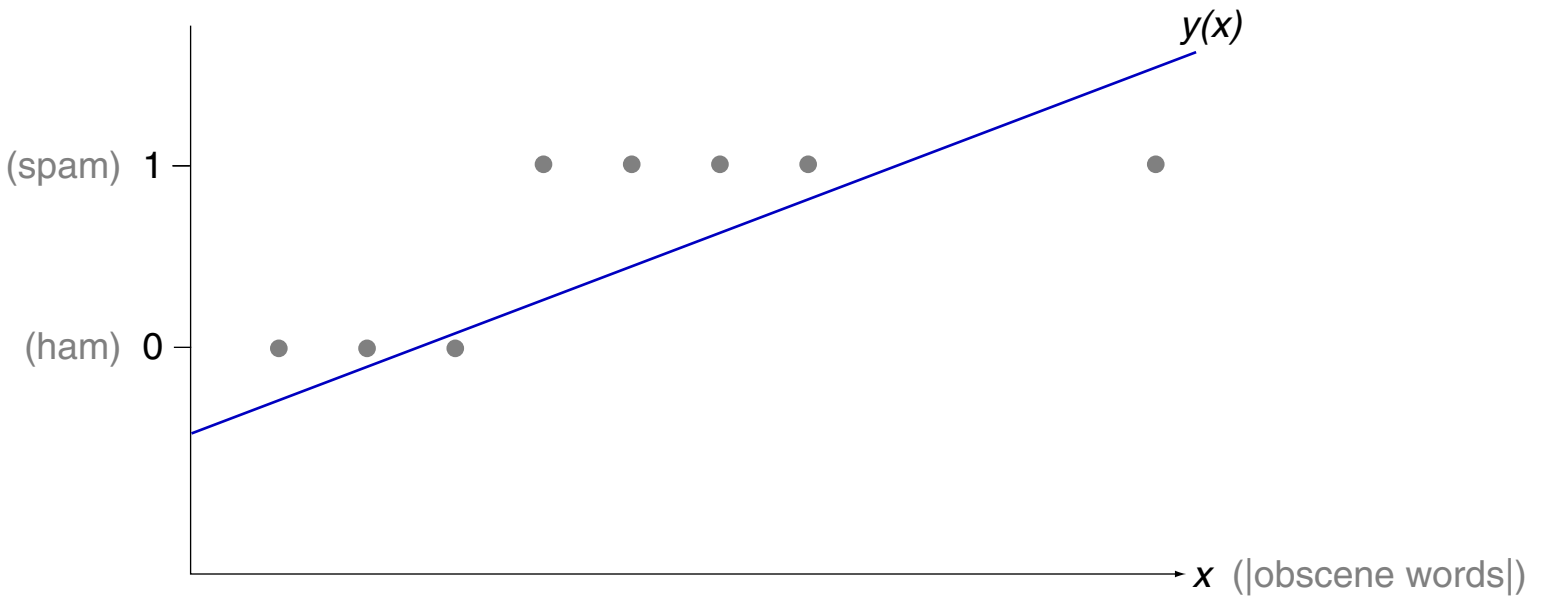


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0.5 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)

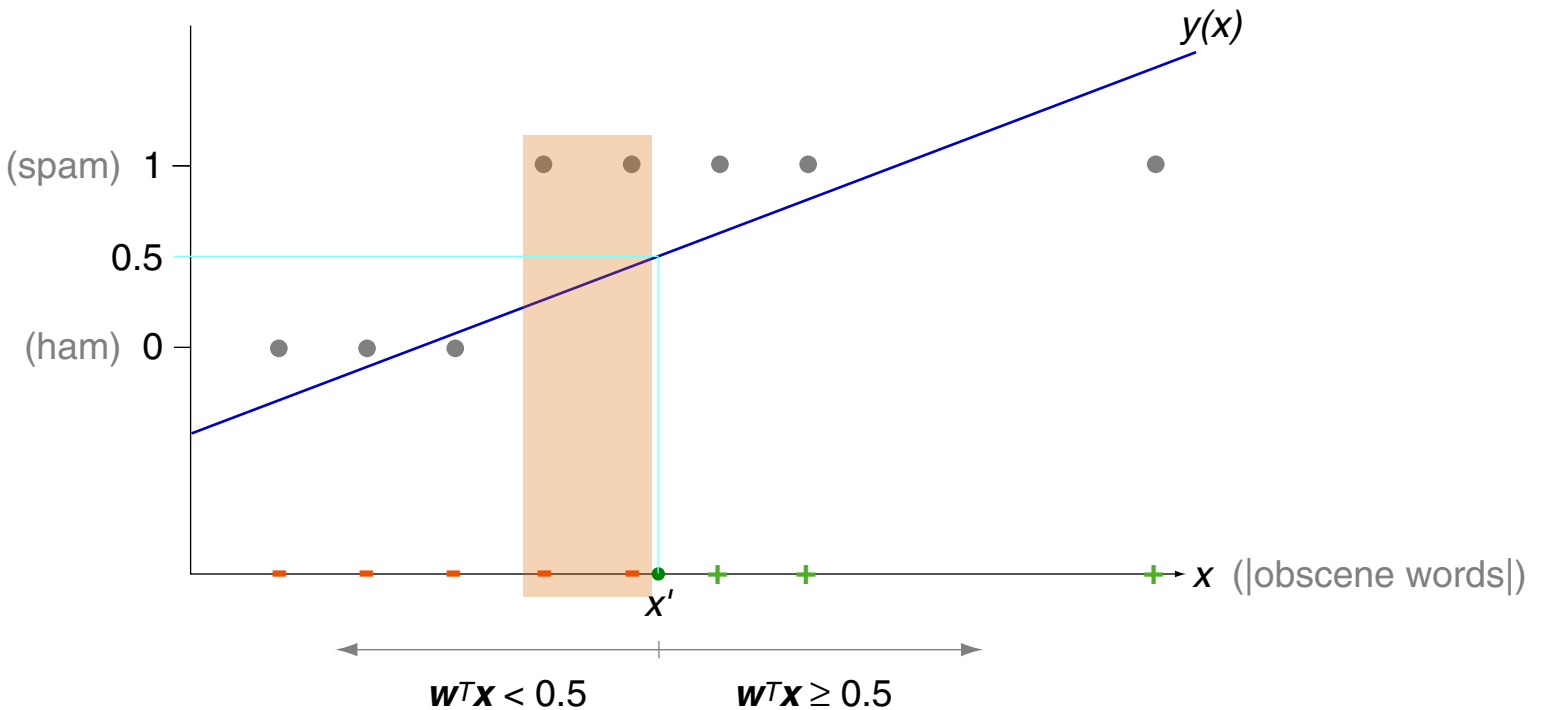


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0.5 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)

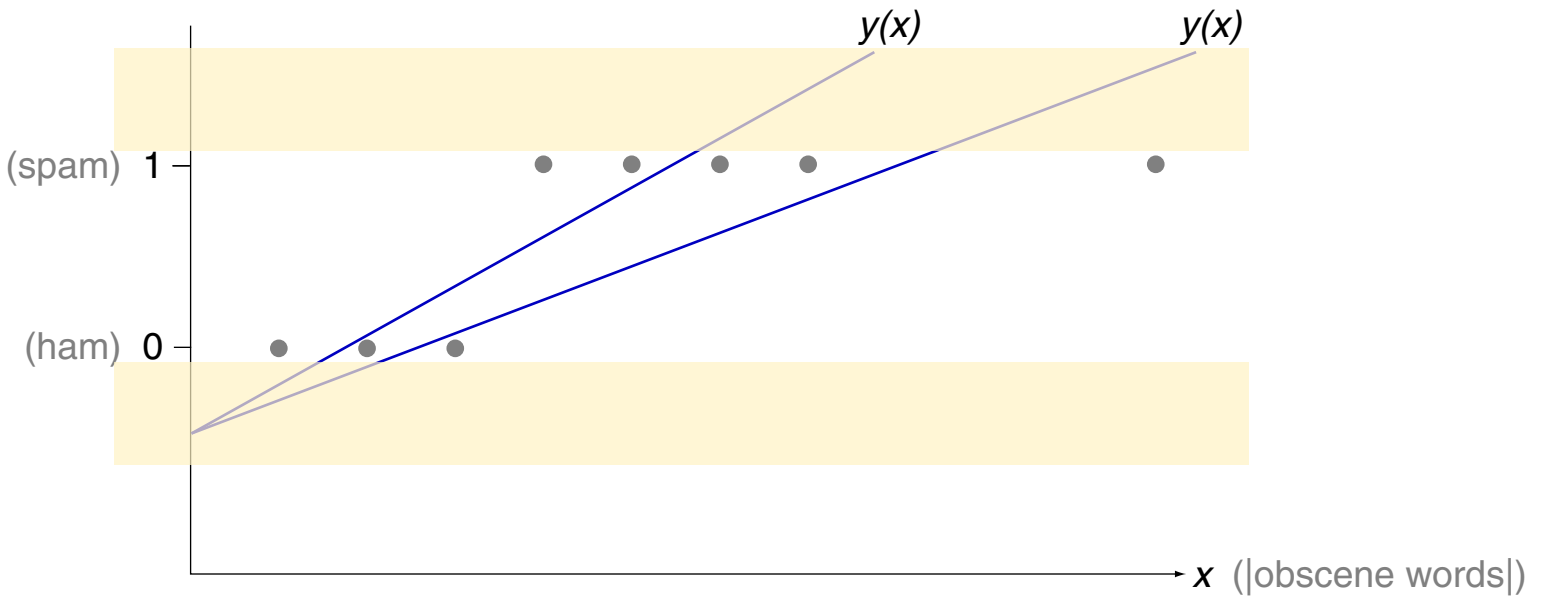


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0.5 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)



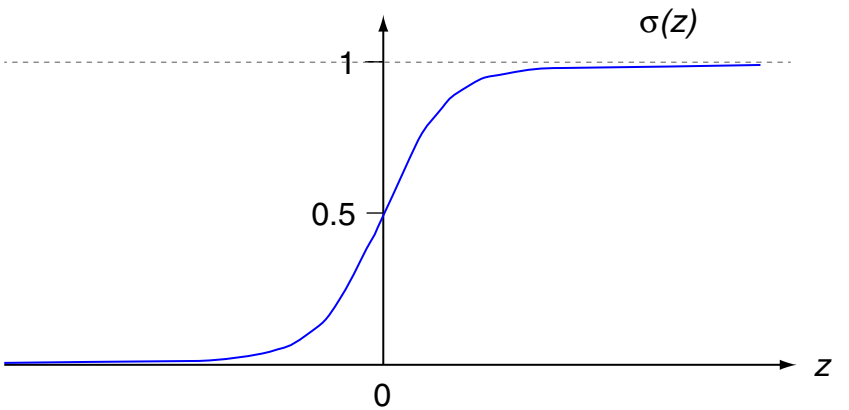
Restrict the range of $y(\mathbf{x})$ to reflect the two-class classification semantics:

$$0 \leq y(\mathbf{x}) \leq 1$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function



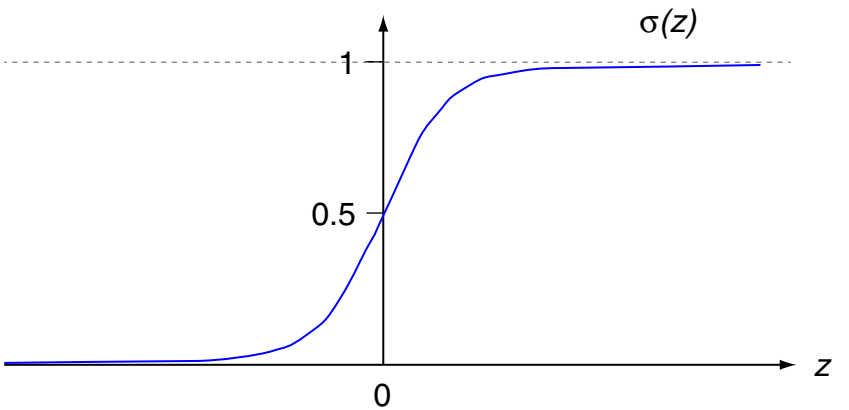
Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function (continued)

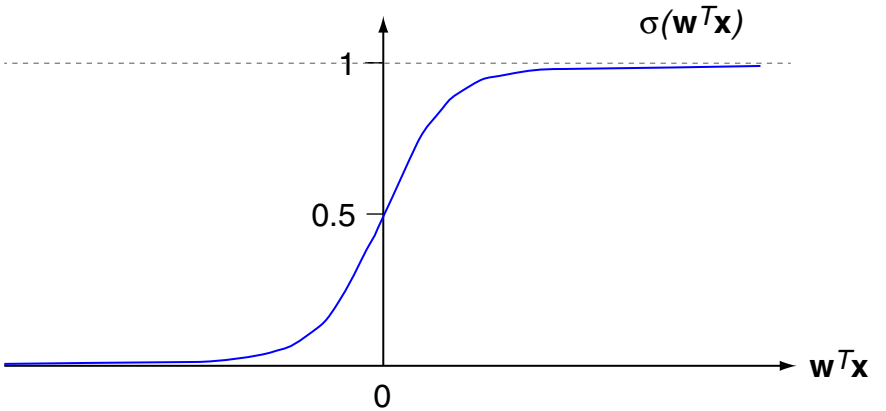


Linear regression	Sigmoid function
$\mathbf{w}^T\mathbf{x}$	$\sigma(z) = \dfrac{1}{1+e^{-z}}$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function (continued)



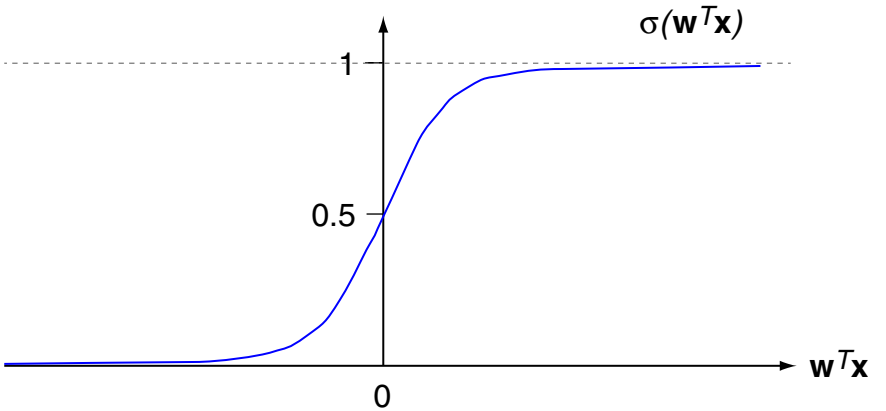
Linear regression | Sigmoid function | Logistic model function

$$\mathbf{w}^T\mathbf{x} \quad \circ \quad \sigma(z) = \frac{1}{1+e^{-z}} \quad \rightsquigarrow \quad y(\mathbf{x}) \equiv \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function (continued)



Linear regression | Sigmoid function | Logistic model function

$$\mathbf{w}^T\mathbf{x} \quad \circ \quad \sigma(z) = \frac{1}{1+e^{-z}} \quad \rightsquigarrow \quad y(\mathbf{x}) \equiv \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

$$y(\mathbf{x}) : \mathbf{R}^{p+1} \rightarrow (0;1)$$

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

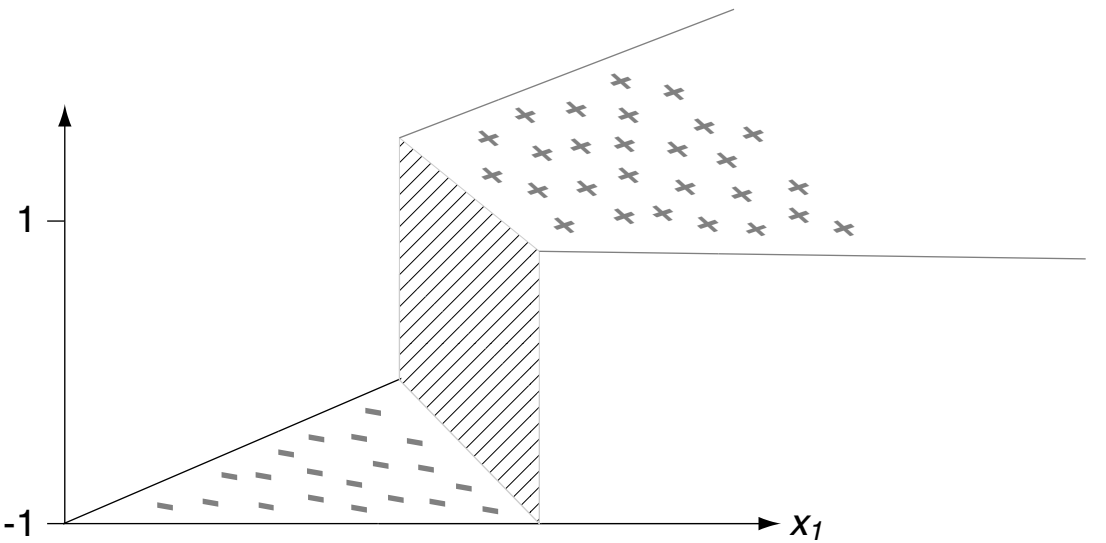
$\rightsquigarrow$ 67% chance that email is spam.

Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

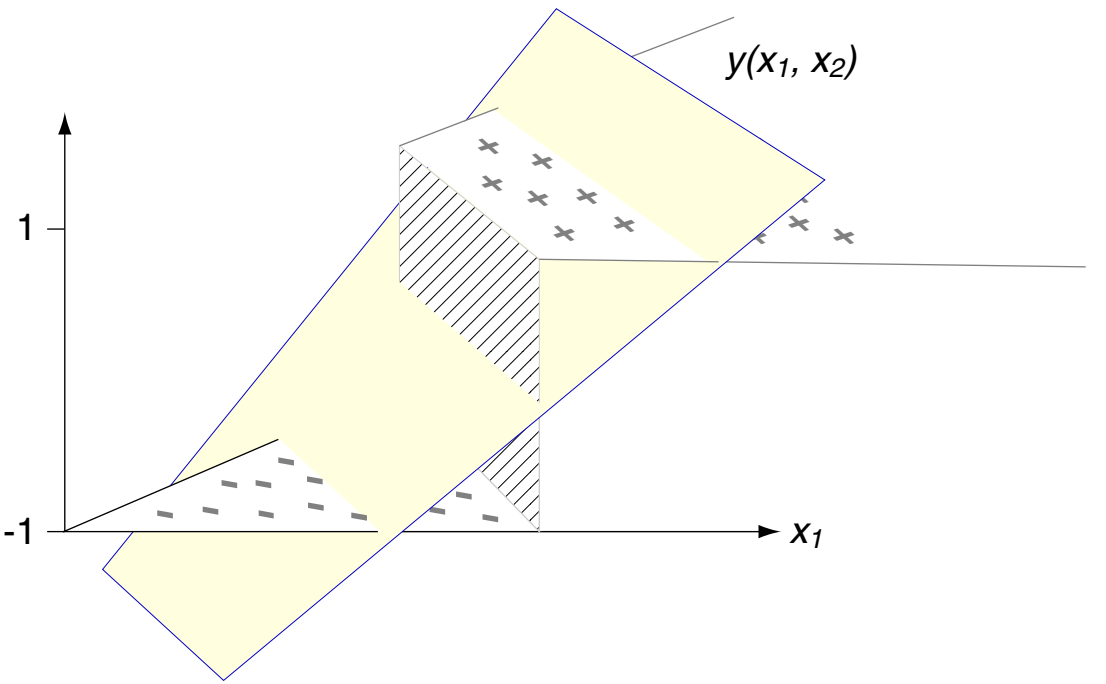
Recap: Linear Regression for Classification (illustrated for $p = 2$)



©STEIN 2005-2020

Logistic Regression

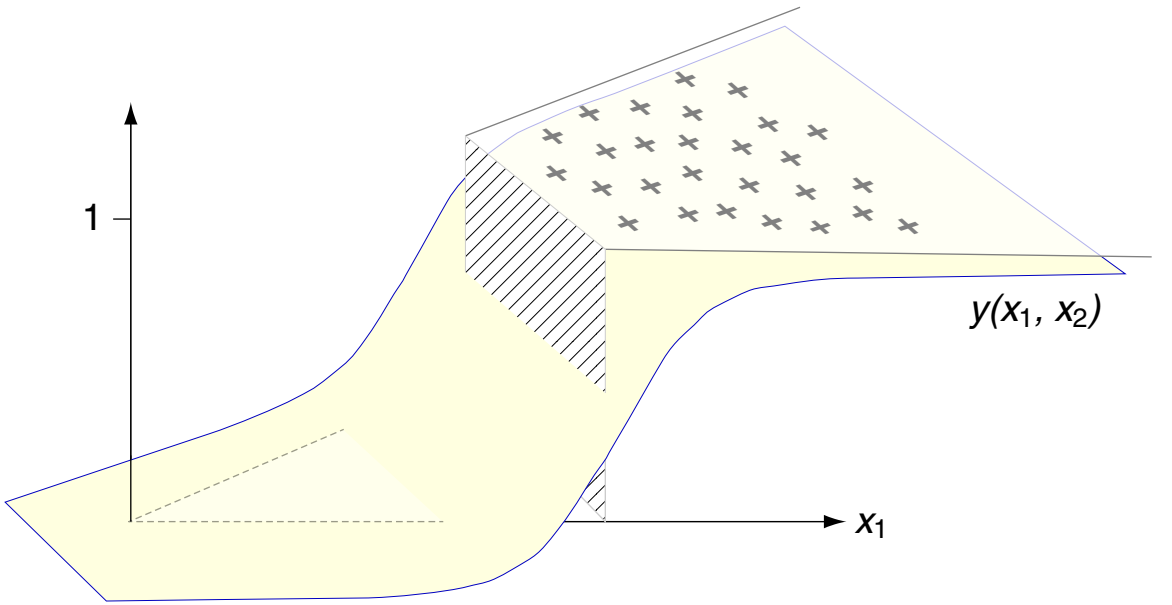
Recap: Linear Regression for Classification (illustrated for $p = 2$)



©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$)

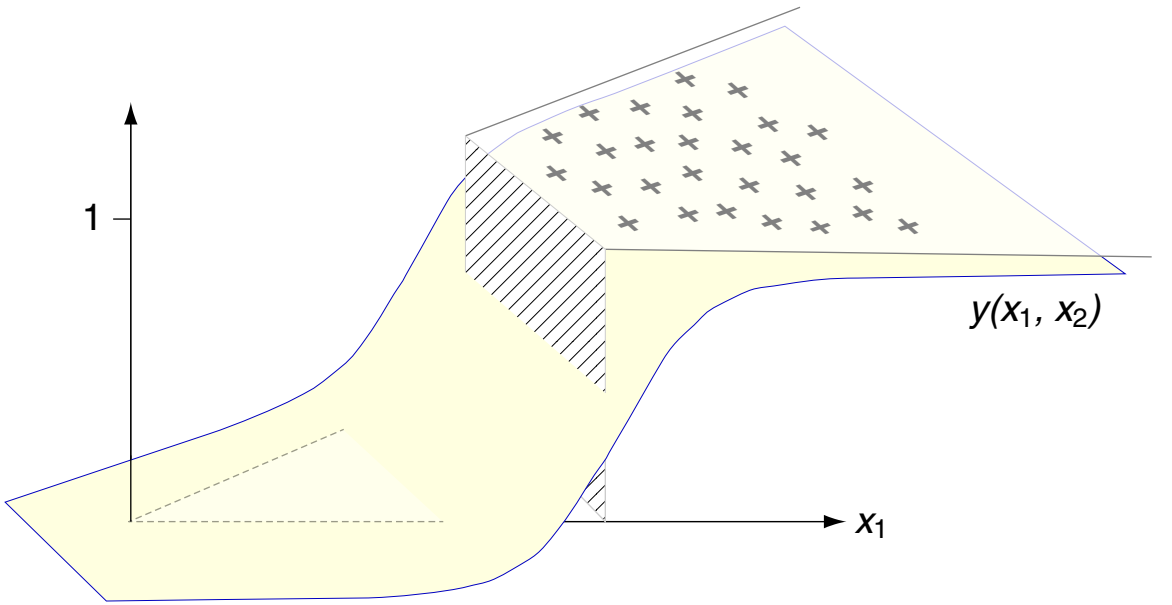


©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$.



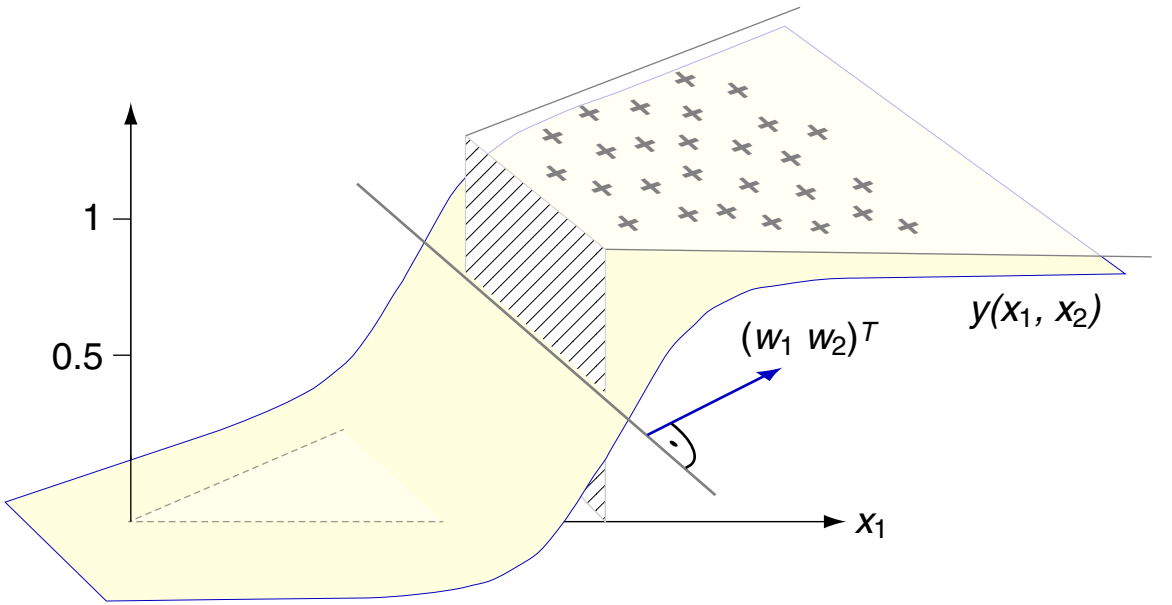
$$y(x_1, x_2) = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$.



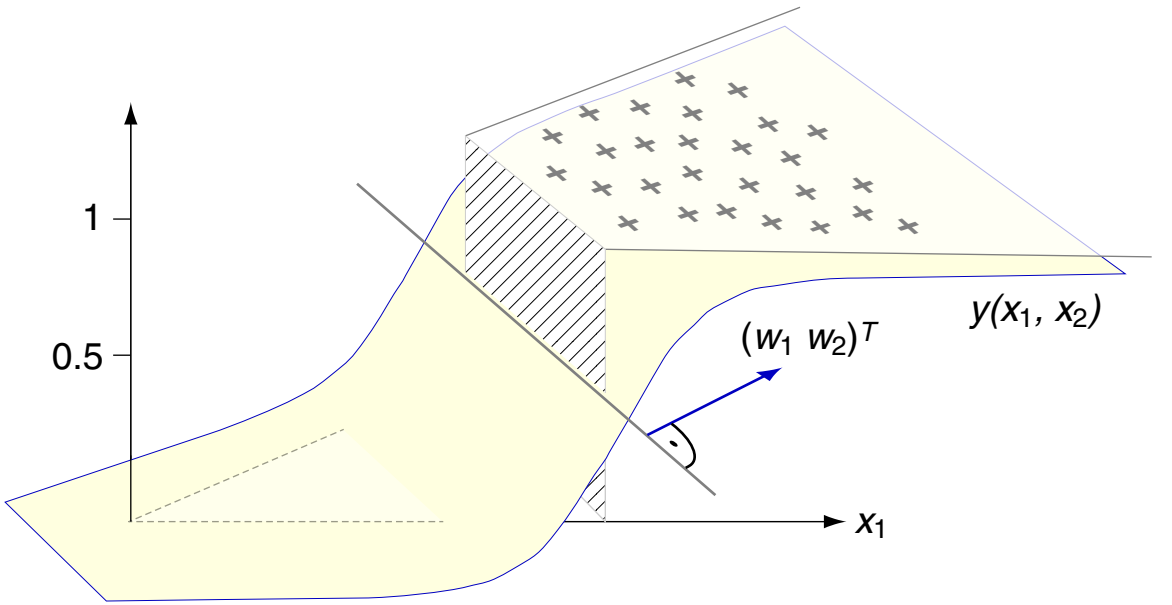
$$y(x_1, x_2) = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



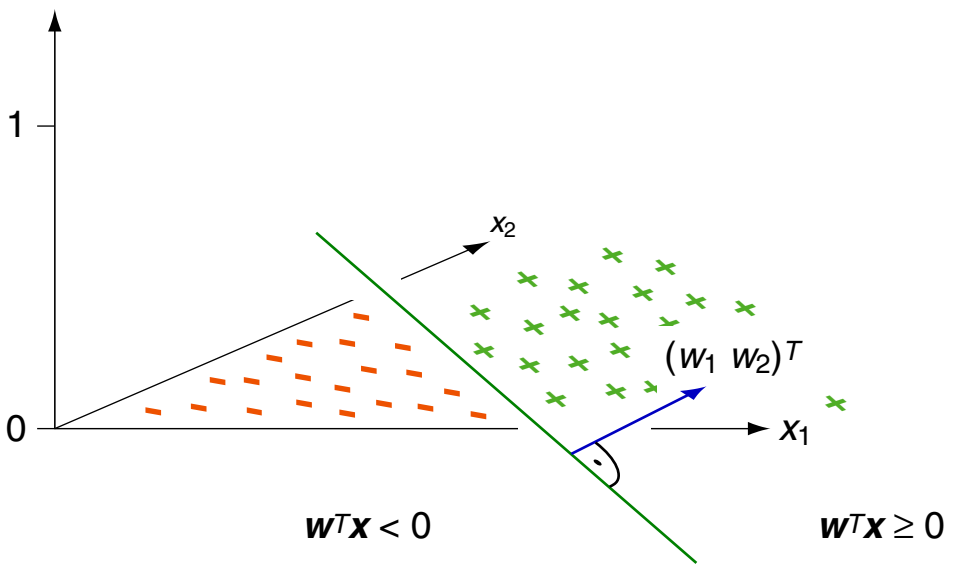
Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



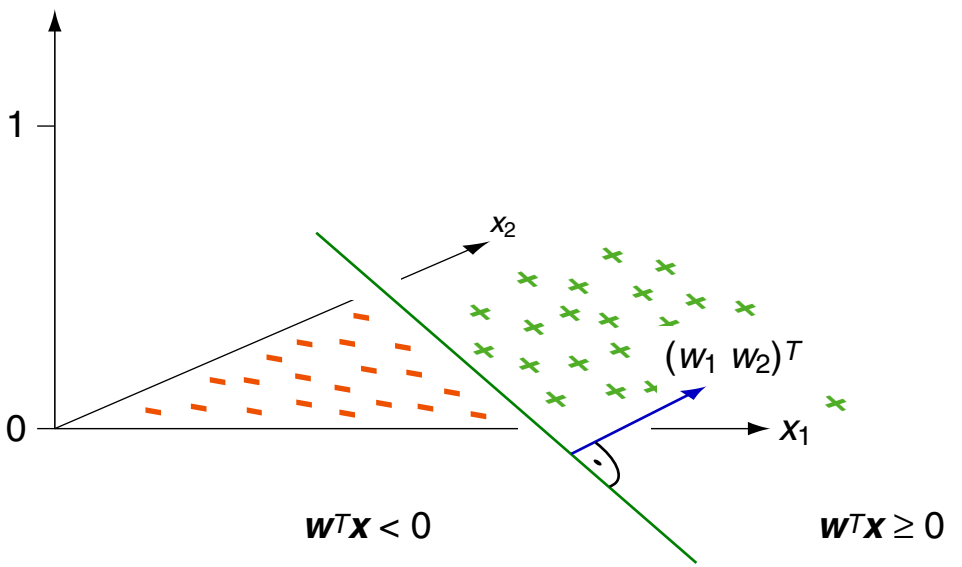
Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 & \Leftrightarrow \mathbf{w}^T\mathbf{x} \geq 0 \\ c(\mathbf{x}) = 0, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 & \Leftrightarrow \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2005-2020

Remarks:

- Observe that under a logistic regression the form of a hypothesis is identical to the hypothesis in linear regression: it is a point, a straight line, a plane, or a hyperplane in the feature space. Similarly, the hypothesis spaces are the same. Hence, the expressiveness, i.e., the complexity of classification problems that can be tackled (or, the effectiveness at which classification problems can be decided) is identical for the two regression approaches.

- Linear regression and logistic regression differ in the way the respective model function parameters, $\mathbf{w}$, are computed. Put another way, the strategy by which the hypothesis space is searched is different. Since both regression methods aim at loss minimization, their different behavior results from their different loss definitions.

- In logistic regression there is no direct method (recall the normal equations in linear regression) to solve the model function $y(\mathbf{x})$ for $\mathbf{w}$.

- By now we have not specified how the parameter vector $\mathbf{w}$ is estimated under the logistic model function.

©STEIN 2005-2020

Logistic Regression

Loss Computation: Linear Regression

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $L_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

©STEIN 2005-2020

Logistic Regression

Loss Computation: Linear Regression (continued)

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) \;=\; I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) \;=\; \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $L_2(y(\mathbf{x}), c(\mathbf{x})) \;=\; (y(\mathbf{x}) - c(\mathbf{x}))^2$

©STEIN 2005-2020

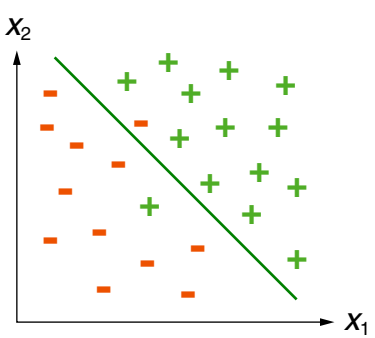
Logistic Regression

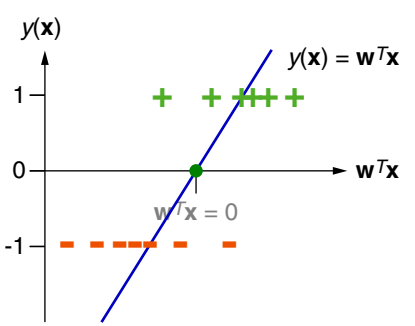
Loss Computation: Linear Regression (continued)

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $L_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

Input space: $y(\mathbf{x})$ over hyperplane distance:





©STEIN 2005-2020

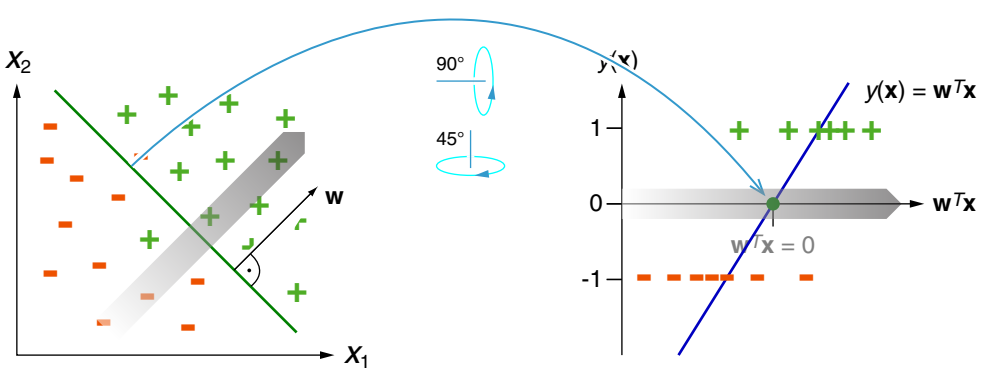
Logistic Regression

Loss Computation: Linear Regression (continued)

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $L_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

Input space: $y(\mathbf{x})$ over hyperplane distance:



©STEIN 2005-2020

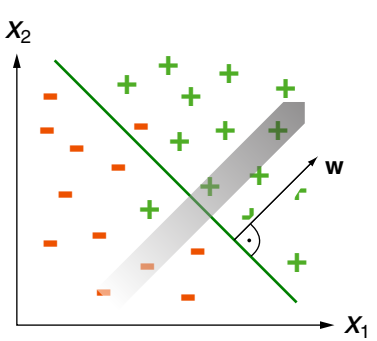
Logistic Regression

Loss Computation: Linear Regression (continued)

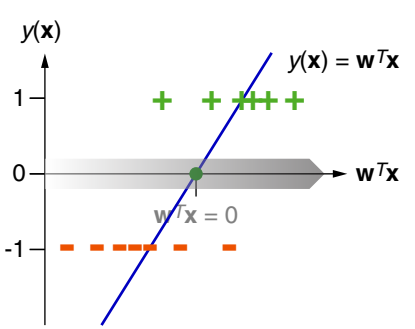
- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $L_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

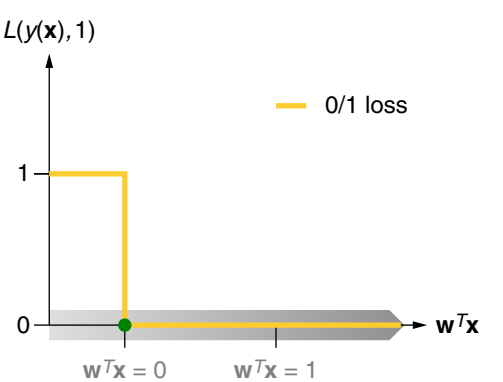
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2005-2020

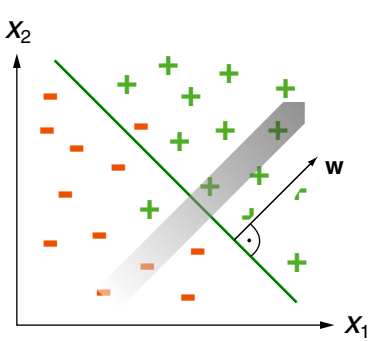
Logistic Regression

Loss Computation: Linear Regression (continued)

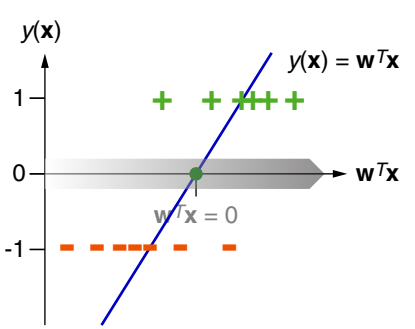
- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $L_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

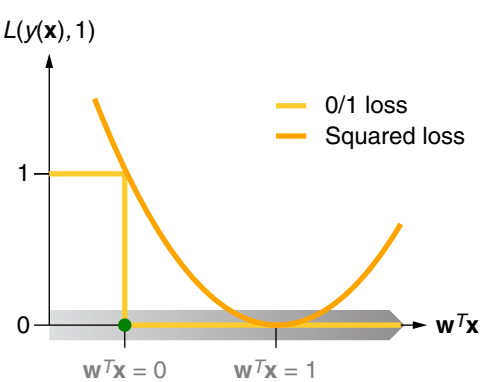
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2005-2020

Remarks:

- We distinguish between the pointwise loss L, which is computed for a single $\mathbf{x}$, and the global loss $\mathcal{L}$, which accumulates the pointwise loss of all $\mathbf{x} \in X$.
- Various loss functions for classification and regression problems have been devised.
- The 0/1 loss computes the misclassification error. Recall in this regard the definition of the true misclassification rate.
- The squared-loss computes the squared residual. Recall in this regard the residual sum of squares.
- As before, I denotes the indicator function.

©STEIN 2005-2020

Logistic Regression

Loss Computation: Logistic Regression

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $L_\sigma(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

©STEIN 2005-2020

Logistic Regression

Loss Computation: Logistic Regression (continued)

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $L_\sigma(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

©STEIN 2005-2020

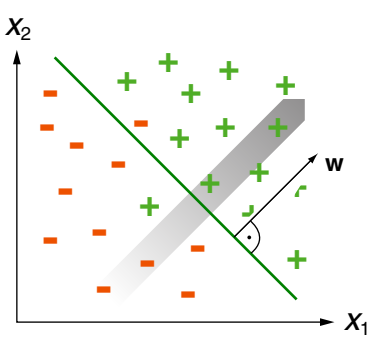
Logistic Regression

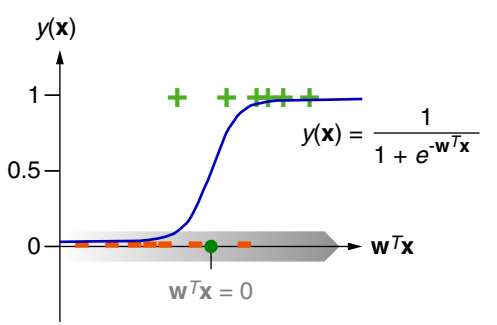
Loss Computation: Logistic Regression (continued)

- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $L_\sigma(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

Illustration:

Input space: $y(\mathbf{x})$ over hyperplane distance:





©STEIN 2005-2020

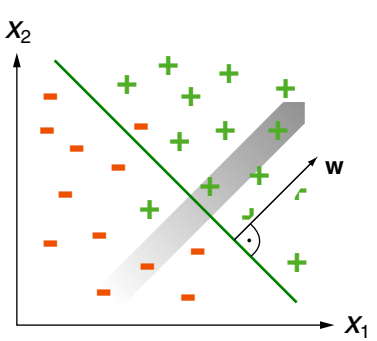
Logistic Regression

Loss Computation: Logistic Regression (continued)

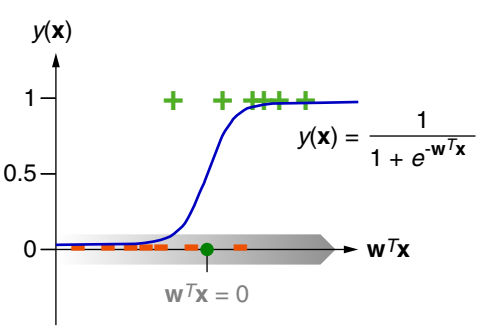
- The pointwise loss, $L(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty charge introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $L_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $L_\sigma(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

Illustration:

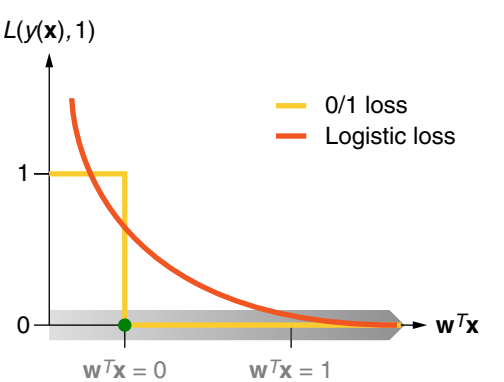
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2005-2020

Remarks:

- The logistic loss can be rewritten: $L_\sigma = -c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1 - c(\mathbf{x})) \cdot \log(1 - y(\mathbf{x}))$
- Observe that loss functions are employed at two places:
  1. For the fitting (the actual regression / optimization / hyperplane search) of the data, where the optimum position of the hyperplane is determined (= a particular hypothesis is selected), and,
  2. for the evaluation of a hypothesis' effectiveness, where the portion of correctly and misclassified examples is analyzed.
- Typically, fitting (optimization) and effectiveness evaluation are done with different loss functions. E.g., logistic regression uses L_σ and $L_{0/1}$ for fitting and evaluation respectively. However, linear regression (not classification) uses RSS (the L_2 loss) for both fitting and evaluation. The basic perceptron learning algorithm uses the misclassification information (the $L_{0/1}$ loss) for both fitting and evaluation.

©STEIN 2005-2020

Chapter ML:III (continued)

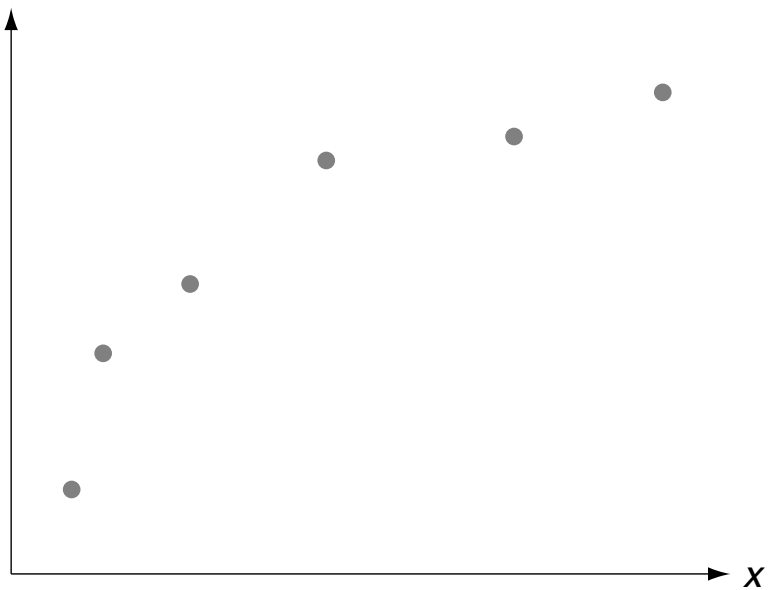
III. Linear Models

- Logistic Regression
- Overfitting
- Regularization

©STEIN 2005-2020

Overfitting

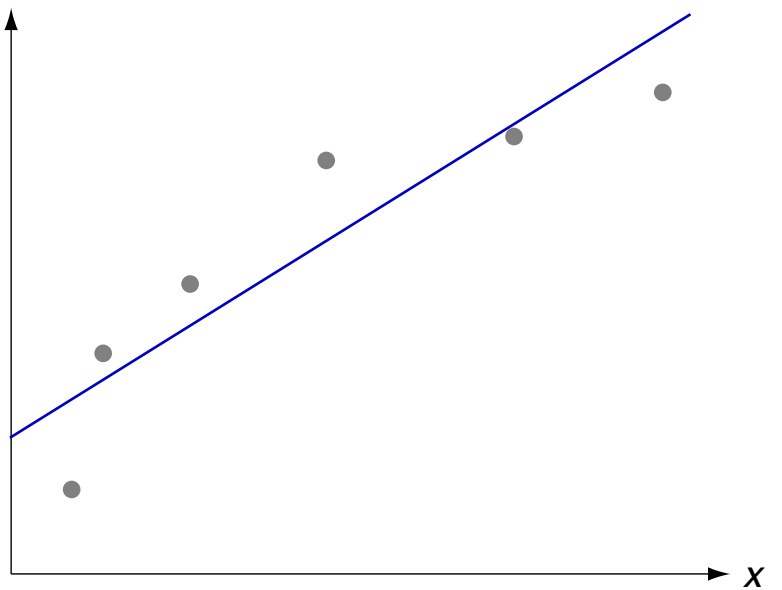
Example: Linear Regression



©STEIN 2005-2020

Overfitting

Example: Linear Regression

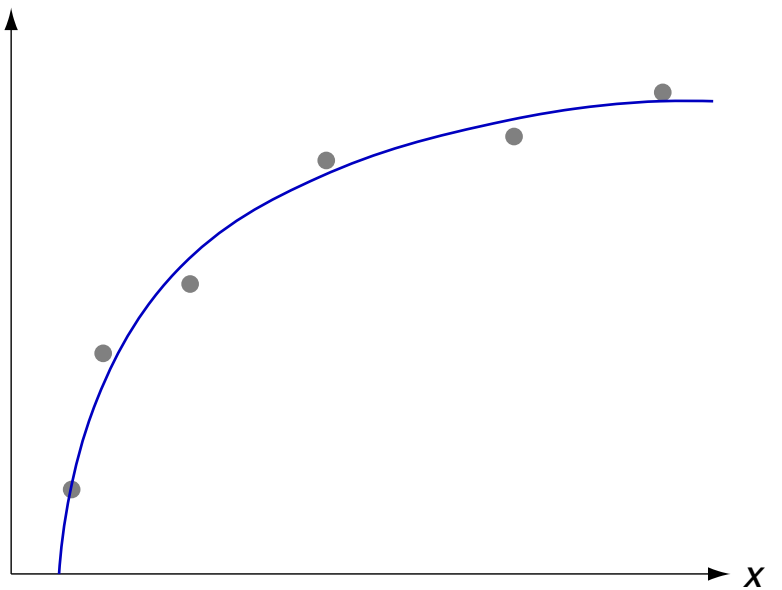


$$y(x) = w_0 + w_1 \cdot x \quad \rightsquigarrow \quad \text{RSS}(w_0, w_1) \gg 0$$

©STEIN 2005-2020

Overfitting

Example: Linear Regression

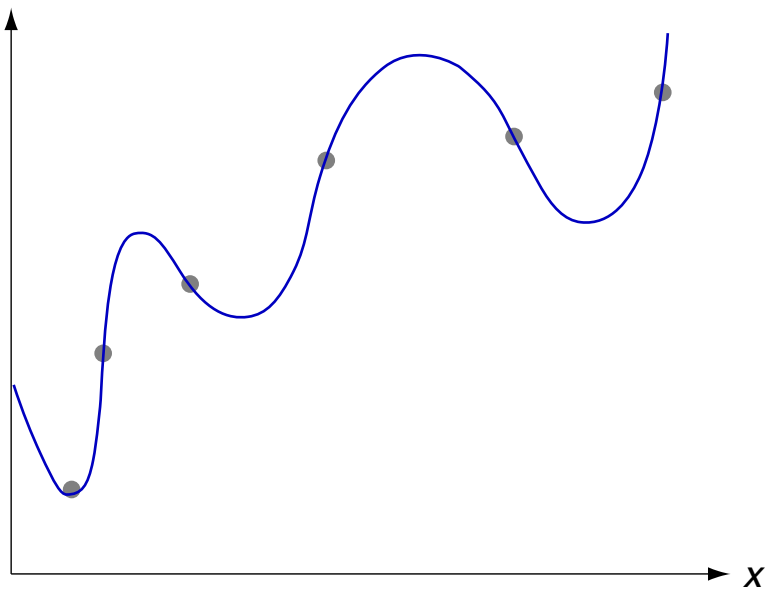


$$y(x) = w_0 + w_1 \cdot x + w_2 \cdot x^2 \quad \rightsquigarrow \quad \text{RSS}(w_0, w_1, w_2) > 0$$

©STEIN 2005-2020

Overfitting

Example: Linear Regression

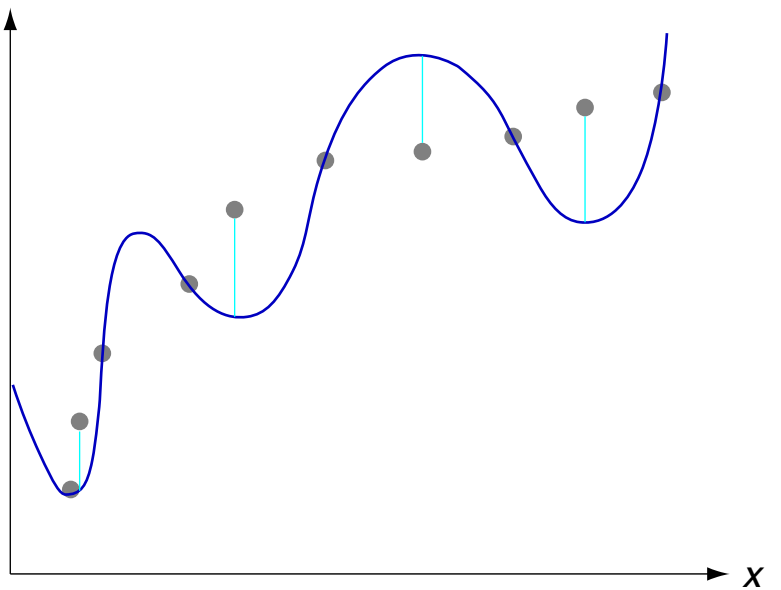


$$y(x) = w_0 + \sum_{i=1}^{6} w_i \cdot x^i \quad \rightsquigarrow \quad \text{RSS}(w_0, w_1, \ldots, w_6) \approx 0$$

©STEIN 2005-2020

Overfitting

Example: Linear Regression

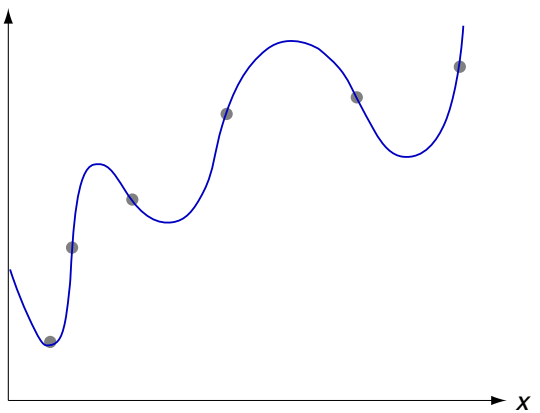


$$y(x) = w_0 + \sum_{i=1}^{6} w_i \cdot x^i \quad \rightsquigarrow \quad \text{RSS}(w_0, w_1, \ldots, w_6) \gg 0$$

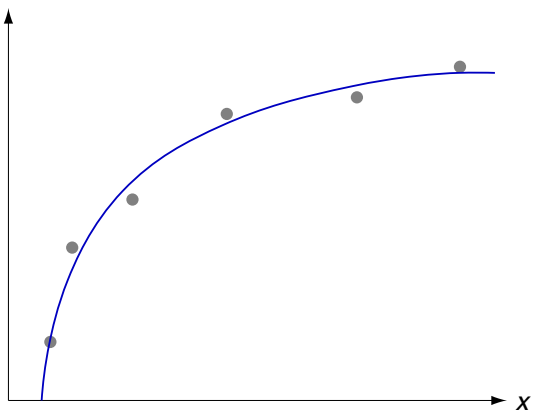
©STEIN 2005-2020

Overfitting

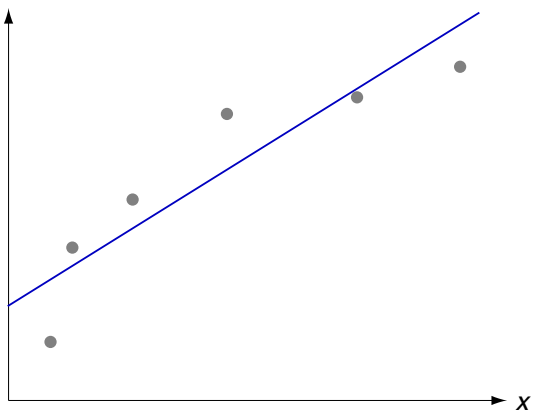
Example: Linear Regression



$\text{RSS}(w_0, w_1, \ldots, w_6) \approx 0$



$\text{RSS}(w_0, w_1, w_2) > 0$



$\text{RSS}(w_0, w_1) \gg 0$

©STEIN 2005-2020

Overfitting

Definition

Model "fitting" is the process of optimizing the parameters of a model function y from a selected hypothesis space to obtain the "best fit" to a sample of data D.

Overfitting literally means "Fitting the data more than is warranted." [Abu Mostafa, 2012]

Causes:

- Too complex model (e.g., too many features, unrelated features).
- Noise in the data (i.e., random fluctuations).
- Too small sample of data.

Effects:

- Low error on training data, but high error on unseen data.
- Reduction / minimization of inductive bias.
- Increase of variance as a result of sensitivity to noise.

©STEIN 2005-2020

Overfitting

Mitigation Strategies

Detection

- **Visualization**
  Works well for dimensionality $p \in [1, 3]$. Higher dimensionalities can only partially be visualized via projection or embedding into lower dimensionalities.

- **Early stopping: Model Selection**
  Let m be the number of training steps, and $y_{\pi_1}, \ldots, y_{\pi_m}$ the models obtained after each step. Indicator of overfitting: The difference $\mathit{Err}_{\mathit{fit}}(y_{\pi_i}) = \mathit{Err}_{\mathit{val}}(y_{\pi_i}) - \mathit{Err}_{\mathit{tr}}(y_{\pi_i})$ increases with i.

Prevention

- **Increasing quantity or quality of the training data D**
  Quantity: More data averages out noise. Quality: Omitting poor examples enables a better fit.

- **Enforcing a higher inductive bias**
  Choosing a hypothesis space that is less complex.

- **Regularization**
  Adjusting the loss function to penalize model complexity.

©STEIN 2005-2020

Regularization

Overview

Let $L(\mathbf{w})$ denote a loss function used to optimize the parameters $\mathbf{w}$ of a model y.

Regularization introduces a trade-off between model complexity and inductive bias:

$$L^*(\mathbf{w}) = L(\mathbf{w}) + R(\mathbf{w}),$$

where λ controls the importance of the regularization term $R(\mathbf{w}) \geq 0$.

Example:

$$y(x) = w_0 + \sum_{i=1}^{6} w_i \cdot x^i; \qquad L(\mathbf{w}) = \text{RSS}(\mathbf{w})$$

$$R(\mathbf{w}) = 0w_0 + 0w_1 + 0w_2 + 1000w_3 + \ldots + 1000w_6$$

©STEIN 2005-2020

Regularization

Regularized Linear Regression

Setting:

- The feature space $X \subseteq \mathbf{R}^p$ is an inner product space.
- $Y \subseteq \mathbf{R}$ is the output space.
- y_i is the ground truth of the feature vector $\mathbf{x}_i \in X$.
- $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subseteq X \times Y$ is a set of examples.

Todo:

- Approximate Y, which is implicitly given via D, with a linear model function and a regularized loss function.

©STEIN 2005-2020

Regularization

Regularized Linear Regression

Setting:

- The feature space $X \subseteq \mathbf{R}^p$ is an inner product space.
- $Y \subseteq \mathbf{R}$ is the output space.
- y_i is the ground truth of the feature vector $\mathbf{x}_i \in X$.
- $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\} \subseteq X \times Y$ is a set of examples.

Todo:

- Approximate Y, which is implicitly given via D, with a linear model function and a regularized loss function.

Examples for linear regression problems:

- What is the credit line of a bank customer?
- What is the price of a house?

©STEIN 2005-2020

Regularization

Regularized Linear Regression

- Given $\mathbf{x}$, predict a real-valued output under a linear model function:

$$y(\mathbf{x}) = w_0 + \sum_{j=1}^{p} w_j \cdot x_j$$

- Vector notation with $x_0 = 1$ and $\mathbf{w} = (w_0, w_1, \ldots, w_p)^T$:

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$$

- Given $\mathbf{x}_1, \ldots, \mathbf{x}_n$, assess goodness of fit as regularized residual sum of squares:

$$\text{RSS}(\mathbf{w}) = \sum_{i=1}^{n} (y_i - y(\mathbf{x}_i))^2 + \lambda \sum_{j=0}^{p+1} w_j^2 = \sum_{i=1}^{n} (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \mathbf{w}^T \mathbf{w} \tag{1}$$

- Estimate $\mathbf{w}$ by minimizing the residual sum of squares:

$$\hat{\mathbf{w}} = \underset{\mathbf{w} \in \mathbf{R}^{p+1}}{\text{argmin}} \ \text{RSS}(\mathbf{w}) \tag{2}$$

©STEIN 2005-2020

Regularization

Regularized Linear Regression

- Let $\mathbf{X}$ denote the $n \times (p+1)$ matrix, where row i is $(1 \;\; \mathbf{x}_i^T)$, $\mathbf{x}_i \in D$.

  Let $\mathbf{y}$ denote the n-vector of outputs in the training set D.

$\rightsquigarrow$ $\text{RSS}(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^T\mathbf{w}$

©STEIN 2005-2020

Regularization

Regularized Linear Regression

- Let $\mathbf{X}$ denote the $n \times (p+1)$ matrix, where row i is $(1 \;\; \mathbf{x}_i^T)$, $\mathbf{x}_i \in D$.

  Let $\mathbf{y}$ denote the n-vector of outputs in the training set D.

$\rightsquigarrow$ $\text{RSS}(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^T\mathbf{w}$

- Minimizing $\text{RSS}(\mathbf{w})$ by a direct method:

$$\frac{\partial\,\text{RSS}}{\partial \mathbf{w}} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w} = 0$$

$$\begin{aligned} \mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) - \lambda\mathbf{w} &= 0 \\ \Leftrightarrow \quad (\mathbf{X}^T\mathbf{X} + \lambda I)\mathbf{w} &= \mathbf{X}^T\mathbf{y} \\ \Leftrightarrow \quad \mathbf{w} &= (\mathbf{X}^T\mathbf{X} + \lambda I)^{-1}\mathbf{X}^T\mathbf{y} \end{aligned}$$

©STEIN 2005-2020

Regularization

Regularized Linear Regression

- Let $\mathbf{X}$ denote the $n \times (p+1)$ matrix, where row i is $(1 \;\; \mathbf{x}_i^T)$, $\mathbf{x}_i \in D$.

  Let $\mathbf{y}$ denote the n-vector of outputs in the training set D.

$\rightsquigarrow$ $\text{RSS}(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^T\mathbf{w}$

- Minimizing $\text{RSS}(\mathbf{w})$ by a direct method:

$$\frac{\partial\,\text{RSS}}{\partial \mathbf{w}} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w} = 0$$

$$\begin{aligned} \mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) - \lambda\mathbf{w} &= 0 \\ \Leftrightarrow \quad (\mathbf{X}^T\mathbf{X} + \lambda I)\mathbf{w} &= \mathbf{X}^T\mathbf{y} \qquad \text{Normal equations.} \\ \Leftrightarrow \quad \mathbf{w} &= (\underbrace{\mathbf{X}^T\mathbf{X} + \lambda I}_{})^{-1}\mathbf{X}^T\mathbf{y} \qquad \text{If } \lambda > 0. \end{aligned}$$

Conditioning the moment matrix $\mathbf{X}^T\mathbf{X}$ [Wikipedia 1, 2, 3]

©STEIN 2005-2020

Regularization

Regularized Linear Regression

- Let $\mathbf{X}$ denote the $n \times (p+1)$ matrix, where row i is $(1 \;\; \mathbf{x}_i^T)$, $\mathbf{x}_i \in D$.

  Let $\mathbf{y}$ denote the n-vector of outputs in the training set D.

$\rightsquigarrow$ $\text{RSS}(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^T\mathbf{w}$

- Minimizing $\text{RSS}(\mathbf{w})$ by a direct method:

$$\frac{\partial\,\text{RSS}}{\partial \mathbf{w}} = -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) + 2\lambda\mathbf{w} = 0$$

$$\begin{aligned} \mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) - \lambda\mathbf{w} &= 0 \\ \Leftrightarrow \quad (\mathbf{X}^T\mathbf{X} + \lambda I)\mathbf{w} &= \mathbf{X}^T\mathbf{y} \qquad \text{Normal equations.} \\ \Leftrightarrow \quad \hat{\mathbf{w}} \equiv \mathbf{w} &= \underbrace{(\mathbf{X}^T\mathbf{X} + \lambda I)}_{\text{Conditioning the moment matrix } \mathbf{X}^T\mathbf{X} \quad \text{[Wikipedia 1, 2, 3]}}{}^{-1}\mathbf{X}^T\mathbf{y} \qquad \text{If } \lambda > 0. \end{aligned}$$

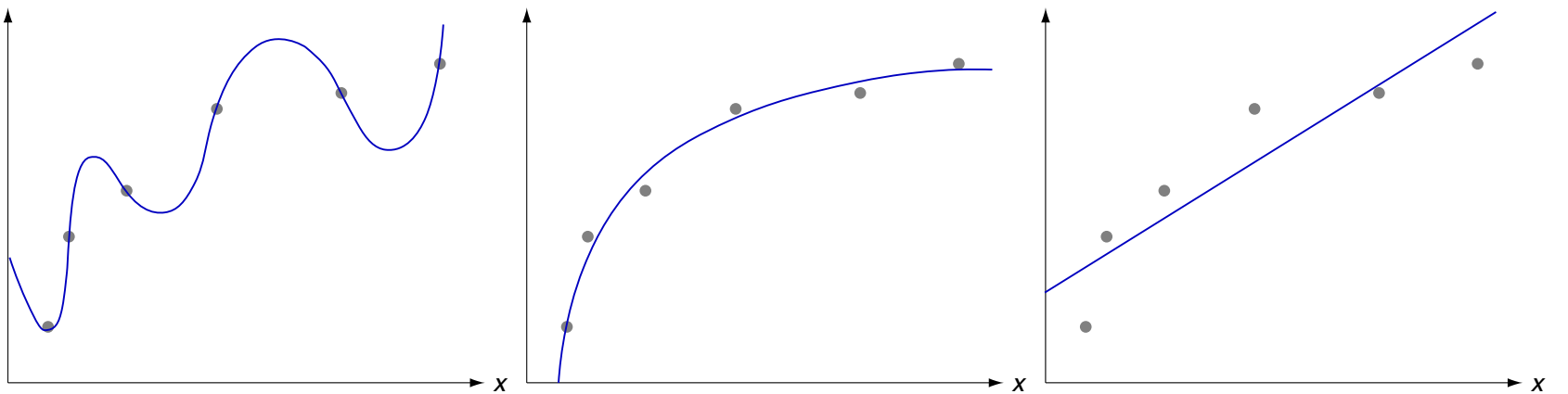
$$\hat{y}(\mathbf{x}_i) = \hat{\mathbf{w}}^T\,\mathbf{x}_i$$

Regression function with least squares estimator $\hat{\mathbf{w}}$.

©STEIN 2005-2020

Regularization

Regularized Linear Regression: Parameter λ



$\lambda = 0 \longrightarrow$

The regularization parameter λ penalizes high weights w_i:

$$y(x) = w_0 + \sum_{i=1}^{p} w_i \cdot x^i = \mathbf{w}^T \mathbf{x}$$

$$\text{RSS}(\mathbf{w}) = \sum_{i=1}^{n} (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \mathbf{w}^T \mathbf{w}$$

©STEIN 2005-2020

Remarks:

- What is underfitting?
- By convention, the parameter w_0 is not included in the regularization term.
- In statistics, the method is known as ridge regression, in machine learning it is known as weight decay, and with multiple independent discoveries, it is also variously known as the Tikhonov-Miller method, the Phillips-Twomey method, the constrained linear inversion method, and the method of linear regularization. It is related to the Levenberg-Marquardt algorithm for non-linear least-squares problems. [Wikipedia]
- The regularization term renders the factor $(\mathbf{X}^T\mathbf{X} + \lambda I)$ invertible, even if the number of features p exceeds the number of examples n in D.

©STEIN 2005-2020